

LLM-Augmented High-Level Route Planning for Autonomous UAV Navigation

Extended Framework: Semantic HRI & Conformal Uncertainty

Prachit Gupta — ModalAI Starling 2 Platform — June 2026

System Overview

The framework separates **semantic reasoning** from **geometric execution**. An LLM acts as a mission planner — converting natural language and scene context into sparse NED-frame waypoints. Classical refinement and verification modules then enforce safety and kinematic constraints before PX4 execution.

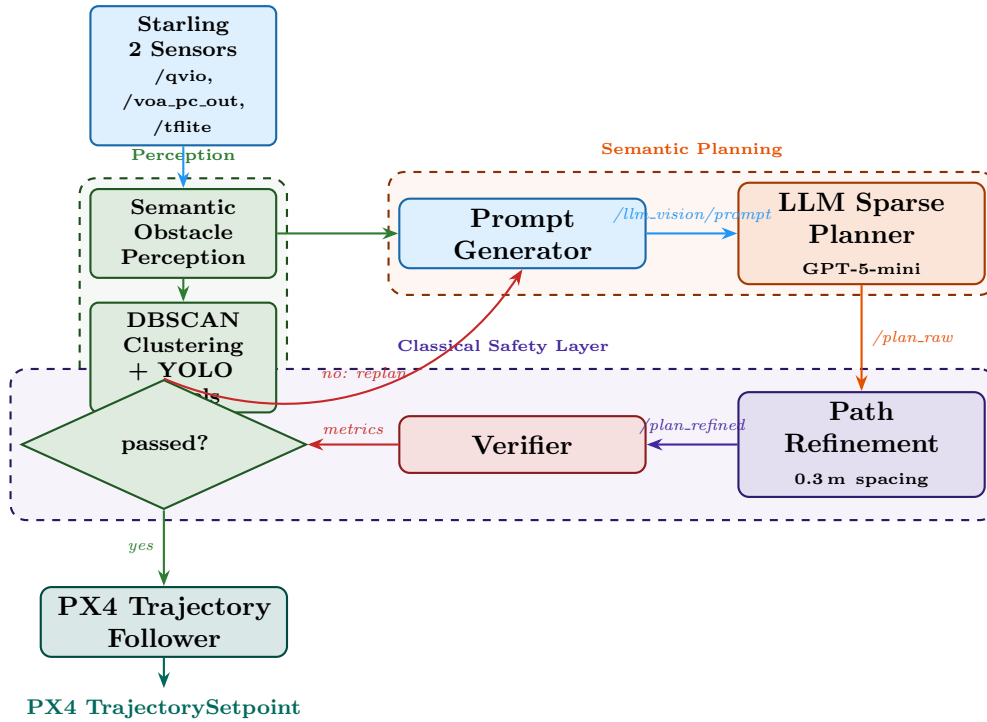


Figure 1: LLM-augmented UAV planning pipeline on the ModalAI Starling2 platform. The LLM is isolated to sparse waypoint generation; all geometric safety checks are deterministic.

Extended Perception Module: Semantic Goal Discovery

The original pipeline assumed a *fixed, coordinate-specified* goal. The extension replaces this with a **natural language goal specification** driven by human interaction. Two perception sub-modes are supported, selectable at runtime. This module is *fully decoupled* from the planning stack — it only updates the goal field inside the prompt generator.

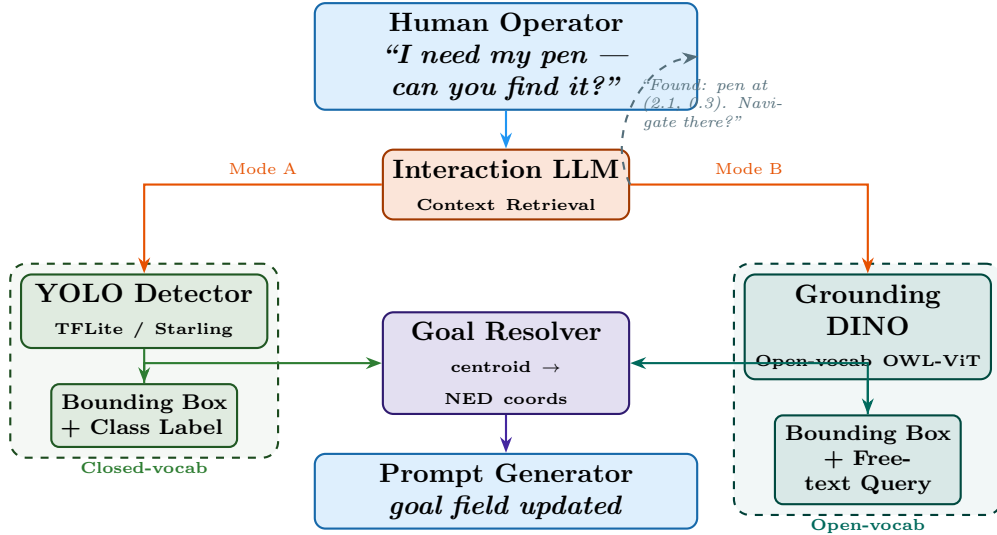


Figure 2: Extended perception module for semantic goal discovery. The interaction LLM retrieves user intent and routes to either the onboard YOLO detector (Mode A) or Grounding DINO (Mode B). The resolved goal centroid is injected into the prompt generator. This module is independent of the planning and verification stack.

Design Principle

The interaction LLM and the planning LLM are **separate roles**. The interaction LLM handles dialogue and object localisation. The planning LLM receives a fully resolved, structured prompt — identical in format to the fixed-goal case — and generates sparse waypoints. Neither LLM has direct access to actuation.

Defensibility of the Planning LLM over Classical Methods

The core question is: *why use an LLM for high-level route planning when A^* , RRT, and data-driven methods already exist?* Four arguments establish defensibility.

Capability	A^* / RRT	DRL / IL	LLM (this work)
Semantic goal specification	None	Partial	Native
Human intent alignment	None	None	Native
Geometric safety	Strong	Strong	Delegated [†]
Novel environments	Map required	Retraining	Zero-shot
Dynamic obstacles	Replan only	If trained	Contextual
Training data requirement	None	Large	Pretrained

Table 1: Capability comparison across planning paradigms. [†]Geometric safety is enforced by the classical refinement and verification modules, not by the LLM directly.

Argument 1 — LLM Decisions Mirror Human Intent

Classical planners optimise a cost function. They have no representation of *why* a human wants to go somewhere or *how* objects relate to each other in context.

Example

Instruction: “Navigate to the chair near the whiteboard, not the one by the door.”

A*: Cannot parse. Requires explicit coordinates.

LLM: Resolves the referential expression using spatial context (“near the whiteboard” constrains which chair), then generates a route that a human operator would recognise as correct.

The LLM encodes social, spatial, and contextual reasoning from human-generated training data. No classical planner replicates this without an exhaustive semantic database.

Argument 2 — LLM is Not Used for Geometric Reasoning

A common objection is that LLMs are spatially unreliable. This is *by design* not a concern here.

Responsibility split

LLM	Sparse route structure, obstacle semantics, human intent, goal specification.
Refinement	Dense interpolation, workspace clamping, obstacle nudging.
Verifier	Collision clearance, kinematic bounds, goal match, monotonic progress.

The LLM produces 2–8 sparse waypoints. Every geometric guarantee is enforced by deterministic, inspectable classical modules. LLM numerical errors are corrected downstream before execution.

Argument 3 — A* and RRT Lack Semantic Awareness

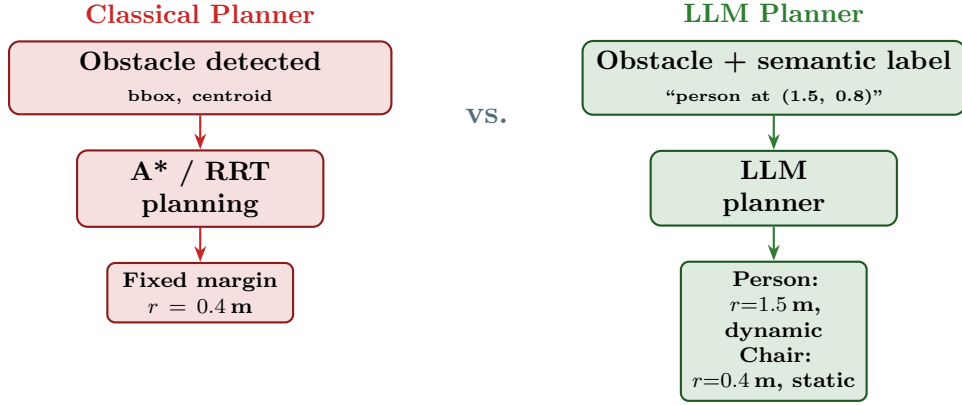


Figure 3: Classical planners apply a uniform margin to all obstacles. The LLM applies *semantically differentiated* margins: wider for humans, tighter for inanimate objects. This is a social/ethical judgement, not a geometric one.

A* and RRT treat all obstacles identically — a chair and a person receive the same clearance radius. They cannot represent the *type* of obstacle, its expected behaviour, or the context in which it appears. An LLM trained on human text encodes these distinctions natively.

Argument 4 — Generalisability over Data-Driven Methods

Deep Reinforcement Learning (DRL) and Imitation Learning (IL) require environment-specific training data. Performance degrades sharply in unseen rooms, novel object configurations, or changed task specifications.

LLM advantage

- **Zero-shot:** operates in any indoor environment without environment-specific training.
- **Few-shot adaptation:** a handful of in-context examples are sufficient to specialise for a new domain.
- **No reward engineering:** the human’s natural language instruction *is* the objective function.

LLM-Augmented RAG for Semantic Safety Constraints

Motivated by LLM-Land [1], the extended pipeline uses a lightweight RAG module to resolve per-obstacle safety margins from semantic labels, replacing the global hardcoded constant.

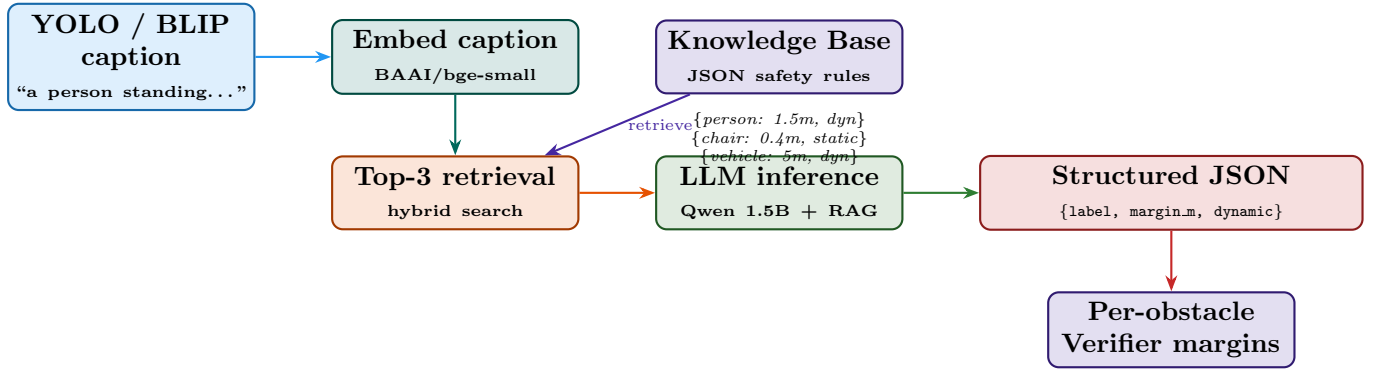


Figure 4: RAG-augmented semantic constraint resolver. Each detected object is individually resolved to a safety margin and dynamic flag. The LLM’s only task is JSON completion over retrieved rules — it cannot hallucinate margins without a retrieved fact to ground on.

Why not a lookup table? A hard-coded table requires exact class-label matches. BLIP and free-text descriptions produce compositional captions (“a group of people near a parked vehicle”) for which no single key exists. The RAG-LLM reads the full sentence, retrieves relevant rules, and composes a justified constraint. Novel object classes are handled zero-shot via world knowledge; a lookup table returns `KeyError`.

Human-in-the-Loop: Closing the Semantic Gap

The LLM naturally brings the human into the planning loop. Rather than a one-way command interface, the system engages in a *minimal dialogue* to resolve goal ambiguity before planning.

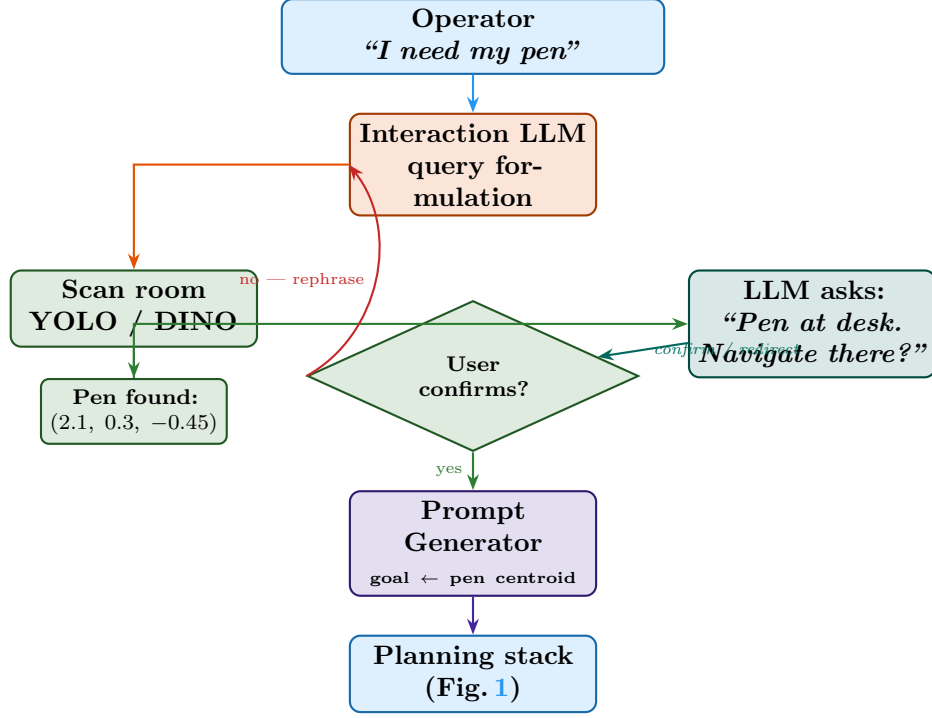


Figure 5: Human-in-the-loop goal resolution. The interaction LLM elicits user confirmation before the goal is committed to the planning stack. This is consistent with MINT [2] and KNOWNO [3]: the system asks *exactly one targeted question* rather than halting or acting on ambiguity.

Uncertainty Quantification via Conformal Prediction

The planning LLM’s outputs are stochastic. Conformal prediction (CP) provides *distribution-free coverage guarantees* over the LLM’s waypoint predictions, enabling the system to quantify when it is confident enough to act autonomously versus when it should request human confirmation.

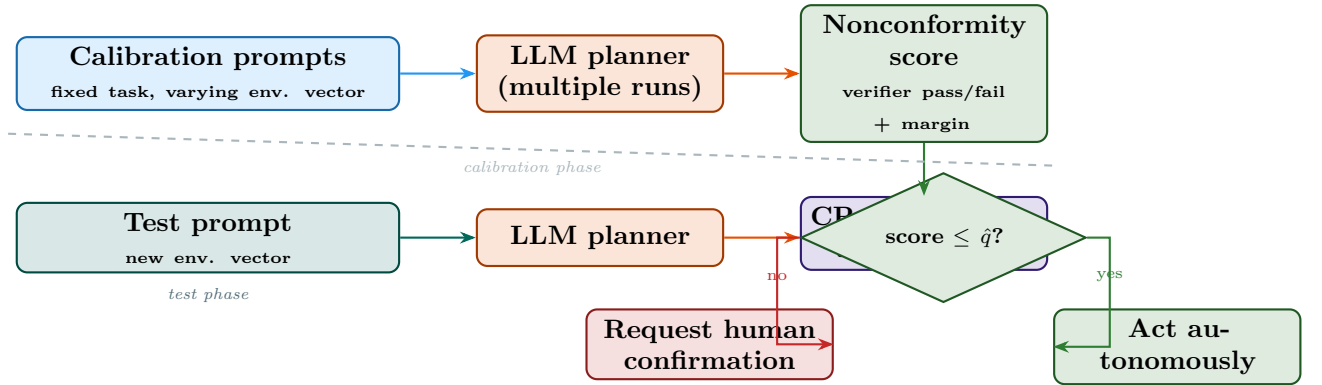


Figure 6: Conformal prediction over LLM waypoint plans. The environmental vector (obstacle layout, object positions) is the exchangeable component across calibration examples. Coverage level $1 - \alpha$ is user-specified (e.g. 0.9).

Calibration data design

- **Fixed:** task instruction type (e.g. “navigate to object X avoiding obstacles”).
- **Exchangeable:** obstacle layout, object positions, room configuration — drawn i.i.d. from the deployment distribution.
- **Ground truth:** verifier pass/fail + minimum clearance serves as the nonconformity score.

Summary

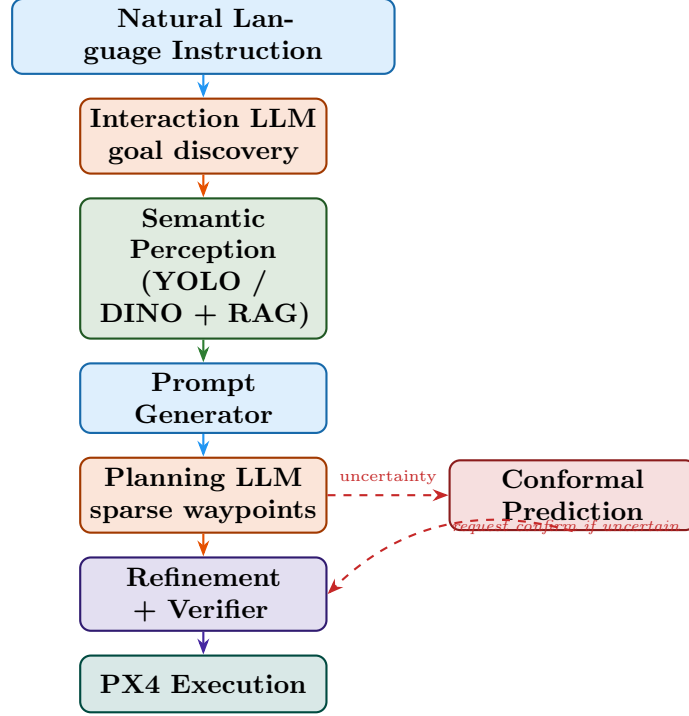


Figure 7: Complete extended pipeline. LLMs handle semantics and human alignment; classical modules enforce geometry; CP provides formal uncertainty guarantees.

Future Work: Scene Graph–Augmented Prompt

Motivation

The current pipeline describes obstacles as an *unordered list* of independent descriptors (label, centroid, bounding box, bearing, distance). This discards relational structure that a human operator would naturally use to reason about a scene — “*the pen is on the desk next to the whiteboard*” conveys more navigational context than listing each object independently.

The hypothesis is that injecting an explicit **spatial relationship graph** into the prompt will ground the LLM’s route decisions in the actual topology of the scene, producing waypoints that better respect object co-locations, adjacencies, and reference landmarks.

Spatial Relationship Extraction

For each pair of detected obstacles (O_i, O_j) , compute a directed spatial relation r_{ij} from their NED centroids and bounding boxes:

Relation	Condition	Example
near	$\ c_i - c_j\ _2 < d_{\text{near}}$	pen near whiteboard
left_of	$\Delta x_{ij} < 0, \Delta y < \epsilon$	chair left of desk
right_of	$\Delta x_{ij} > 0, \Delta y < \epsilon$	door right of whiteboard
in_front	$\Delta y_{ij} < 0$	obstacle in front of UAV
behind	$\Delta y_{ij} > 0$	shelf behind desk
between	O_k centroid inside convex hull(c_i, c_j)	corridor between chairs
adjacent	bounding boxes within margin δ	laptop adjacent to pen

Table 2: Spatial relations computed from NED centroids and bounding box geometry. Thresholds d_{near} , ϵ , δ are environment-calibrated.

Prompt Augmentation

The relationship graph is serialised as a compact natural language paragraph appended to the existing obstacle list in the prompt:

Augmented prompt fragment

Obstacles: [chair at (1.2, 0.5), pen at (2.8, 0.2), whiteboard at (2.7, 0.0)]

Spatial relationships:

- pen is adjacent to whiteboard (dist 0.22m)
- pen is right_of chair (dx = +1.6m)
- whiteboard is in_front of UAV (dy = -0.45m)
- chair is between UAV and pen

Mission: Navigate to the pen.

This gives the LLM an explicit navigational scaffold — the route must pass the chair before reaching the pen/whiteboard cluster — rather than requiring it to infer topology from raw coordinates.

Expected Benefit

Hypothesis

LLM-generated sparse waypoints will show improved obstacle avoidance and reduced verifier failures when the prompt includes relational context, particularly in scenes with clustered objects or referential instructions (“the pen near the whiteboard”). The ablation is: same scene, same LLM, prompt with vs. without the relationship paragraph.

Conformal Prediction: Dataset Design

What Is the Environmental Vector?

For conformal prediction to provide coverage guarantees, the calibration examples must be **exchangeable** — drawn i.i.d. from the same distribution as test inputs. The environmental vector is the component of the prompt that varies across scenes while the task type stays fixed.

Environmental vector $\mathbf{e}$

Component	Representation
Obstacle set	$\{(\text{label}_k, x_k, y_k, w_k, h_k, \text{dist}_k, \text{bearing}_k)\}_{k=1}^K$
Relationship graph	$\{(O_i, r_{ij}, O_j, d_{ij})\}$
UAV start pose	(x_0, y_0, ψ_0) in NED
Goal description	free-text noun phrase (e.g. “the pen”)
Workspace bounds	$[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$

Fixed across all calibration examples: the task instruction template (“navigate to X while avoiding obstacles”), the LLM model and temperature, and the Pydantic response schema.

Independence from Hardware and Perception

Dataset generation is **fully decoupled** from the Starling testbed and the perception stack. The perception module’s only output that matters downstream is a structured obstacle dictionary — label, centroid, bounding box, size, distance, bearing. For calibration, this dictionary is *sampled synthetically* rather than measured from sensors.

What is bypassed

Not needed	Starling hardware, PX4, depth camera, QVIO
Not needed	DBSCAN clustering, point-cloud filtering
Not needed	YOLO / TFLite inference
Required	LLM API call (GPT-5-mini)
Required	<code>prompt_generator.py</code>
Required	<code>refinement.py</code> + <code>verifier.py</code>

A synthetic scene is generated by sampling obstacle centroids uniformly from the workspace $[0, 4] \times [0, 4]$ m, drawing labels from the YOLO class vocabulary, and computing size/bearing/distance from per-class priors. The resulting dict is identical in schema to what `semantic_obstacle_perception.py` publishes on `/llm_vision/semantic_obstacles` — so `prompt_generator.py` receives it without modification. The generation loop is pure Python, runnable on any machine with API access.

Dataset Structure

Each calibration example is a tuple:

$$\mathcal{D}_{\text{cal}} = \{(\mathbf{e}^{(i)}, \mathbf{w}^{(i)}, y^{(i)})\}_{i=1}^N$$

Field	Type	Description
$\mathbf{e}^{(i)}$	structured dict	Synthetic environmental vector
$\mathbf{w}^{(i)}$	list of (x, y, z)	LLM-generated sparse waypoints
$\hat{\mathbf{w}}^{(i)}$	list of (x, y, z)	Refined + verified trajectory
$y^{(i)}$	bool	Verifier passed flag
$s^{(i)}$	float	Nonconformity score
$c_{\min}^{(i)}$	float (m)	Minimum obstacle clearance
$\Delta_{\text{goal}}^{(i)}$	float (m)	Final waypoint-goal distance
$\text{mono}^{(i)}$	bool	Monotonic goal progress
$n_{\text{replan}}^{(i)}$	int	Replanning attempts
graph_used	bool	Scene graph in prompt?

Table 3: Schema for one calibration example. All fields are produced by the software pipeline; no hardware is required.

How to collect: Run the planning stack (prompt generator $\rightarrow$ LLM $\rightarrow$ refinement $\rightarrow$ verifier) in a headless Python loop across N randomised synthetic scenes. Vary obstacle count (1–5), types, positions, and goal objects. Target $N \geq 200$ for stable quantile estimates at $\alpha = 0.1$.

Example Dataset (5 Rows)

Table 4 shows five illustrative rows. The environmental vector is summarised as a readable scene description; the full JSON is stored per row in the dataset file.

ID	Scene description	Goal	y	$c_{\min}$ (m)	Δ_{goal} (m)	mono	n_r	s
1	chair (1.2, 0.5) <i>pen adjacent whiteboard</i> <i>pen right_of chair</i>	pen (2.8, 0.2)	P	0.82	0.03	✓	0	0.02
2	person (1.5, 0.8) whiteboard (3.1, 0.0) <i>whiteboard in_front UAV</i>	whiteboard (3.1, 0.0)	P	0.46	0.02	✓	0	0.09
3	chair (1.0, 0.4) chair (1.8, 0.6) <i>chairs adjacent</i>	laptop (3.0, 1.0)	F	0.18	0.04	✓	1	1.21
4	person (0.8, 0.3) person (2.1, 0.7) <i>persons between UAV-goal</i>	pen (3.5, 0.5)	F	0.57	0.05	×	2	1.60
5	whiteboard (2.0, 0.0) <i>(no relations: single obstacle)</i>	chair (3.8, 0.9)	P	0.91	0.02	✓	0	0.01

Table 4: Five example calibration rows. **P** = verifier passed, **F** = failed. Scene descriptions are human-readable summaries of the environmental vector; the full obstacle dict and relationship graph are stored as JSON. Italicised lines show the relationship graph entries. n_r = replan count. s = composite nonconformity score ($\lambda_1=1.0$, $\lambda_2=0.5$, $\lambda_3=0.3$, $\lambda_4=0.1$).

Reading the table. Row 1 is an easy open scene: LLM routes cleanly around a single chair to reach the pen, clearance 0.82 m, score near zero. Row 3 illustrates the failure mode: two adjacent chairs create a narrow corridor; the LLM’s waypoint clips inside the safety margin, verifier rejects once, replan succeeds, but the elevated score ($s = 1.21$) flags this as an uncertain scene type. Row 4 fails monotonicity: two persons between UAV and goal cause the LLM to generate a detour that temporarily increases the goal distance; two replans are needed, producing the highest score in the batch. These high-score rows are exactly what CP uses to set $\hat{q}_{1-\alpha}$: at test time, a similarly cluttered scene triggers a human confirmation request before the trajectory is latched.

Nonconformity Score

The nonconformity score $s^{(i)}$ measures how *surprising* the LLM’s output is relative to the calibration distribution.

Composite nonconformity score

$$s^{(i)} = \lambda_1 \cdot \mathbf{1}[y^{(i)} = \text{fail}] + \lambda_2 \cdot \max(0, m - c_{\min}^{(i)}) + \lambda_3 \cdot \Delta_{\text{goal}}^{(i)} + \lambda_4 \cdot n_{\text{replan}}^{(i)}$$

where $m = 0.40$ m is the safety margin. Suggested weights: $\lambda_1 = 1.0$, $\lambda_2 = 0.5$, $\lambda_3 = 0.3$, $\lambda_4 = 0.1$.

At coverage level $1 - \alpha$, the CP quantile is:

$$\hat{q}_{1-\alpha} = \text{Quantile}\left(\{s^{(i)}\}_{i=1}^N; \frac{\lceil (N+1)(1-\alpha) \rceil}{N}\right).$$

At test time, if $s^{\text{test}} > \hat{q}_{1-\alpha}$, the system requests human confirmation rather than acting autonomously.

Labels

Label	Values	Use
$y^{(i)}$	{pass, fail}	Binary CP over planning success
$c_{\min}^{(i)}$	$\mathbb{R}_{\geq 0}$ (m)	Regression CP for clearance
$n_{\text{replan}}^{(i)}$	$\mathbb{Z}_{\geq 0}$	Ordinal label for replanning cost
$s^{(i)}$	$\mathbb{R}_{\geq 0}$	Composite nonconformity score

Table 5: Labels per calibration example.

Scene graph ablation flag

Add a binary flag `graph_used` $\in \{0, 1\}$ to each example to enable direct comparison of nonconformity scores with and without the spatial relationship paragraph in the prompt. This operationalises the scene graph hypothesis as a controlled experiment within the CP framework.

References

- [1] S. Cai, Y. Wu, L. Zhou, *LLM-Land: Large Language Models for Context-Aware Drone Landing*, arXiv:2505.06399, 2025.

- [2] *Reasoning Knowledge-Gap in Drone Planning via LLM-based Active Elicitation*, AAAI Symposium Series, 2026.
- [3] M. Ren et al., *Robots That Ask for Help: Uncertainty Alignment for LLM Planners*, arXiv:2307.01928, 2023.
- [4] Wang, He, Kantaros, *S-ATLAS: Probabilistically Correct Language-Based Multi-Robot Planning Using Conformal Prediction*, IEEE RA-L, 2025.
- [5] *FM-Planner: Foundation Model Guided Path Planning for Autonomous Drone Navigation*, arXiv:2505.20783, 2025.